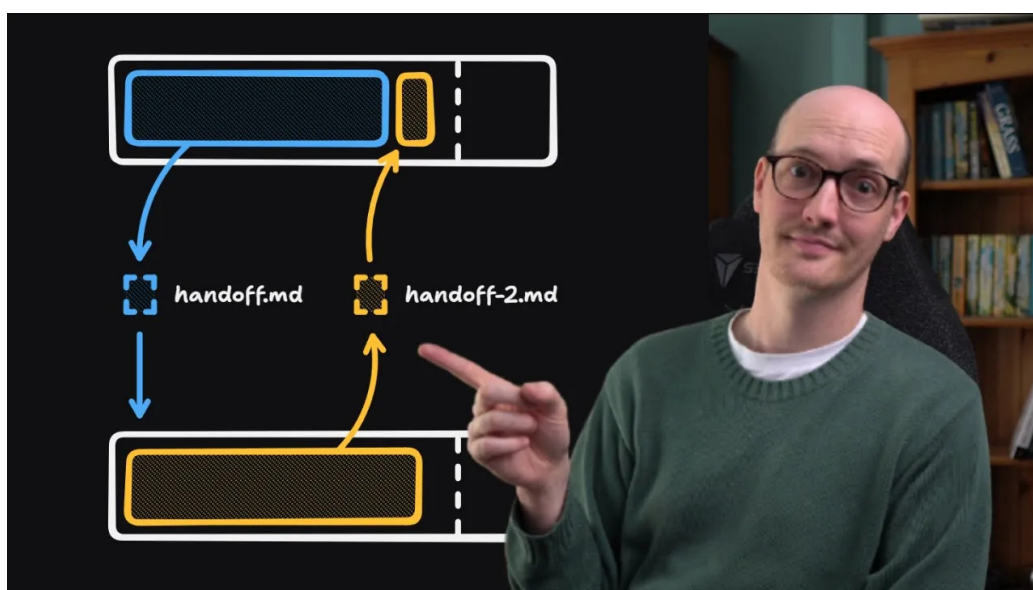


课程笔记

/handoff is my new favourite skill

Matt Pocock & Codex

2026 年 5 月 31 日



视频作者/频道: Matt Pocock

发布日期: 2026-05-21

视频时长: 12:24

视频链接: <https://www.youtube.com/watch?v=dtAJ2d0d3ko>

目录

1 从一个“小到不像技能”的动作说起	3
1.1 观察：一个反复出现的代理动作	3
1.2 handoff 的第一版定义	3
1.3 为什么“小”反而值得固定成技能	4
1.4 本章小结	4
2 上下文窗口、smart zone 与 dumb zone	5
2.1 上下文窗口不能当免费仓库	5
2.2 smart zone / dumb zone：注意力变浑的直觉	5
2.3 /compact 如何把长会话压回可工作区	6
2.4 compact 的沉积层效应	7
2.5 本章小结	8
3 handoff 与 compact 的真正分界	8
3.1 问题：旁支任务会污染主会话	8
3.2 compact 的局限：它会接管当前进度	9
3.3 handoff 的机制：只切出相关切片	9
3.4 两个会话并行：保持主线纯净	10
3.5 本章小结	10
4 在 grilling 中把旁支问题移出去	10
4.1 grilling 是什么	10
4.2 一次 Q2 里的 handoff	11
4.3 为什么说明“交出去的原因”很关键	12
4.4 范围收缩带来的反作用	12
4.5 本章小结	13
5 prototype 往返：手搓一个轻量 subagent	13
5.1 known unknowns 与必须看见的东西	13
5.2 把困难部分交给 prototype	13
5.3 从 prototype 再 handoff 回 planner	14
5.4 DIY subagent 的结构	15
5.5 跨代理可移植性	16
5.6 本章小结	17

6 技能设计：让 handoff 文档短命、聚焦、可执行	17
6.1 suggested skills: 给下一位代理预装工作方式	17
6.2 用指针替代重复内容	18
6.3 临时目录与信息安全	19
6.4 参数即下一会话的焦点	20
6.5 总结与延伸	20

1 从一个“小到不像技能”的动作说起

1.1 观察：一个反复出现的代理动作

Matt Pocock 在视频一开头没有先讲一个宏大的 agent 架构，他从一个很小的习惯说起：几周前，他发现在和编码代理协作时反复做同一个动作。这个动作看起来很聪明，但小到让他一开始觉得“不值得做成一个 skill”（00:00–00:12）。这里的关键不在于动作有多复杂，关键在于它是否反复出现、是否每次都要重新解释、是否能被固定成可复用的 workflow。

他给出的背景很具体：自己一直在思考怎样把编码直觉和实践包装成 reusable skills，他的 skills repo 在录制时接近 100,000 stars（00:10–00:24）。这句话不需要被理解成权威背书，更像一个线索：Matt 观察 skill 的方式，通常来自“我在真实工作里一再手动这样做”。当某个提示词、某段流程、某个交接动作反复出现，它就开始有资格成为 skill。

Skill 的判断标准

一个动作是否值得做成 skill，不看它是否高级，而看它是否稳定、可复现、可迁移。稳定，意味着每次触发条件差不多；可复现，意味着代理能按同一套步骤做；可迁移，意味着换一个任务或会话后仍然有用。

1.2 handoff 的第一版定义

视频里，Matt 把这个动作命名为 handoff skill。第一版定义很朴素：它把当前会话里与继续工作有关的信息压缩成一个 Markdown 文件，然后把这个文件交给另一个新会话，让新代理能接着做（00:24–00:37）。这一章先把 handoff 理解成“交接文档”就够了：它像工程交班时写给下一位值班工程师的短报告，说明现在做到了哪里、下一步该关注什么、哪些文件或决定值得看。

name	handoff
description	Compact the current conversation into a handoff document for another agent to pick up.
argument-hint	What will the next session be used for?

Write a handoff document summarising the current conversation so a fresh agent can continue the work. Save to the temporary directory of the user's OS - not the current workspace.

Include a "suggested skills" section in the document, which suggests skills that the agent should invoke.

Do not duplicate content already captured in other artifacts (PRDs, plans, ADRs, issues, commits, diffs). Reference them by path or URL instead.

Redact any sensitive information, such as API keys, passwords, or personally identifiable information.

If the user passed arguments, treat them as a description of what the next session will focus on and tailor the doc accordingly.

图 1: 画面显示 handoff 技能说明：名称为 handoff，描述为把当前对话压缩成交接文档，正文高亮了“让 fresh agent 继续工作”以及“保存到用户操作系统临时目录、不要保存到当前 workspace”的规则。¹

屏幕上出现的 skill 文本进一步把它落到文件层面：它要求写一个 handoff document，总结当前对话，使 fresh agent 可以继续工作；保存位置是用户操作系统的临时目录，而非当前 workspace（00:44–

¹ 视频画面时间区间：00:00:35–00:01:10。

00:57)。这个保存位置很重要。放在临时目录，说明 handoff 文件是一次性工作交接材料；放进项目仓库，很容易变成过期文档，后来的人分不清它是源事实、历史草稿，还是某次临时实验的残留。

视频还给出一个具体落点：这个 skill 位于 skills 仓库的 `productivity/handoff` 下 (00:37–00:44)。这个路径细节说明它被 Matt 当成日常工作流工具，而非某个项目里的临时脚本。

本章的 handoff 定义

handoff 是一个面向新代理会话的 Markdown 交接文档。它的职责是把当前会话中足以继续某个任务的状态、意图、线索和下一步压缩出来，并以临时文件的形式传给 fresh agent。

这里有两个容易混在一起的概念。第一，handoff document 是具体文件，通常是 Markdown。第二，handoff skill 是生成这个文件的可复用流程。前者像交班单，后者像“每次交班时必须写清楚哪些栏目”的制度。Matt 一开始觉得这个制度太小，结果放进 skills 文件夹后发现自己用了很多次 (00:57–01:03)。

1.3 为什么“小”反而值得固定成技能

这个例子的现实感在于：很多 agent 工作流的瓶颈，并不来自模型不会做惊天动地的大事，而来自人类需要反复补充同一类边界。比如“把刚才相关内容整理给另一个会话”“别把临时文件写进仓库”“让新代理知道下一步关注什么”。这些句子单独看都很普通，重复十几次之后就变成成本。

Matt 在 01:03–01:16 预告了这支视频的路线：解释为什么设计这个 skill、它有什么用、它和某些 harness 里的 builtin tool 如 compact 如何比较、以及它怎样用于 grilling sessions。换句话说，handoff 的价值要放在 agent 会话管理里看：它处理的核心不在代码语法，焦点是“一个会话的工作状态如何被另一个会话接住”。

常见误解：小动作不等于低价值

在 agent 协作里，小动作经常是工作流的铰链。铰链本身不豪华，但门能不能顺畅开合取决于它。handoff 的设计价值就在这里：它把“请把这段上下文整理给下一位代理”从临时口头指令变成固定动作。

因此，本章不急着把 handoff 抬高成复杂系统。先记住它的最小形态：一个短命、聚焦、可转交的 Markdown 文件。后面几章会继续扩展它和上下文预算、分支任务、prototype、跨代理协作之间的关系。

1.4 本章小结

第一章的重点只有三条。第一，handoff 来自一个反复出现的真实代理动作，而非抽象设计。第二，它的第一版形态是 Markdown 交接文档，让 fresh agent 能继续某个明确任务。第三，它应该写到操作系统临时目录，因为它服务于一次工作交接，不能悄悄变成项目长期文档。

2 上下文窗口、smart zone 与 dumb zone

2.1 上下文窗口不能当免费仓库

Matt 从 01:36 开始解释 handoff 为什么有必要，第一站是上下文窗口。上下文窗口可以先用一句话定义：它是当前代理生成回答时能够看到并处理的活动 token 范围，里面包括用户消息、代理回复、工具调用结果、文件编辑记录、错误输出、压缩摘要等会话材料。它像一张工作台，但技术上更准确地说，它是一段被模型注意力机制处理的 token 序列。

视频中的例子很工程化：在一个 coding session 里，人和代理对话，代理执行 tool calls，修改文件，返回结果；这些材料都会一点点填进上下文窗口（01:42–02:01）。材料越多，窗口越满。窗口越满，代理要在更多 token 之间建立关联，注意力关系会更拥挤。

先用直觉说：把上下文窗口想成一张桌面。桌面上只有当前任务的设计稿、报错和两三个文件名时，人能快速定位；桌面上堆满十轮尝试、旧计划、废弃分支、重复日志和历史争论时，即使桌面还没物理爆满，找东西也会慢下来。模型的上下文也有类似压力。

可以用一个很轻的符号表示这个概念：

$$C = (m_1, m_2, \dots, m_n), \quad |C| \approx \sum_{i=1}^n \text{tokens}(m_i)$$

- C ：当前会话的上下文窗口，也就是模型本轮生成时可见的活动材料。
- m_i ：上下文里的第 i 个消息、工具结果、编辑记录、摘要或其他片段。
- n ：当前上下文中片段的数量。
- $|C|$ ：当前上下文的大致 token 规模。
- $\text{tokens}(m_i)$ ：片段 m_i 占用的 token 数。

上下文窗口的精确定义

上下文窗口是模型在一次生成中可见的活动 token 空间。它不能充当项目记忆库，也不能充当无限草稿箱；它更像当前回合必须搬上工作台的材料集合。材料越多，模型越需要在更多 token 之间分配注意力。

这个定义解释了为什么“模型标称支持很长上下文”仍然不等于“长会话一直同样聪明”。Matt 提到他使用的 Claude Code harness 里上下文窗口很大，字幕显示为 1 million tokens（02:01–02:09）。但他紧接着说，实际使用中存在 smart zone 和 dumb zone。

2.2 smart zone / dumb zone：注意力变浑的直觉

smart zone 和 dumb zone 是 Matt 的口语化说法，并非严格 benchmark 指标。它描述的是长会话中的体感变化：早期上下文负载低，代理更专注；后期上下文堆积多，回答更容易变钝、变散、变笨。



图 2: 画面显示上下文窗口示意条: 左侧蓝色填充区域接近虚线分界, 分界上方标有 120k, 右侧红字标出 Dumb Zone。²

Matt 的解释抓住了注意力机制的直觉: 早期上下文里 token 少, 需要计算和权衡的 token 关系少, attention relationships 没有那么 strained, 代理的注意力也没有那么 diffuse (02:09–02:33)。随着对话继续推进, 工具输出、文件编辑、尝试路径和中间判断不断进入 C , 模型要在更大的材料堆里判断什么重要, 回答质量就可能下降。

他说得很直: 对话越发展, 代理的回答会越来越 dumb; 虽然可以一路堆到 800,000 tokens, 他自己没有走到那么远, 因为大约到 120k token mark 时, 他已经开始感觉进入 dumb zone (02:33–02:53)。这个数字不能被当成所有模型、所有任务、所有 harness 的固定阈值, 但它是一个有用的工程经验: 窗口上限和高质量工作区间是两件事。

不要把 advertised context window 当成有效工作区间

标称 1 million tokens 说明系统可能容纳这么长的上下文; 它不保证第 900k token 处的任务判断仍像第 20k token 处一样锋利。Matt 给出的 120k 是个人使用体感, 并非通用定律, 但它提醒使用者必须预算上下文。

这里还藏着一个用户很容易漏掉的点: 长上下文的风险不只来自“放不下”, 也来自“放得下但变浑”。传统程序的数组容量满了会报错; agent 会话的上下文变浑时, 症状更隐蔽, 表现为重复问已经回答的问题、忽略关键约束、把旧方案当新方案、在局部文件上过度操作。它不一定崩溃, 但会悄悄降低决策质量。

2.3 /compact 如何把长会话压回可工作区

当会话开始顶到 dumb zone, Matt 提出的问题很具体: 怎样恢复对话, 怎样继续推进, 同时保持 smart (03:08–03:21)。他的答案是 compact。

在本章里, compact 定义为: 把一个很大的会话总结成较小的状态摘要, 让同一条工作线可以在更短的上下文起点上继续。它通常会保留引用过的文件、用户说过的关键内容、对话的总体语气或目标, 并把这些内容作为一个小 nugget 放到新会话开头 (03:21–04:08)。

²视频画面时间区间: 00:01:35–00:03:05。

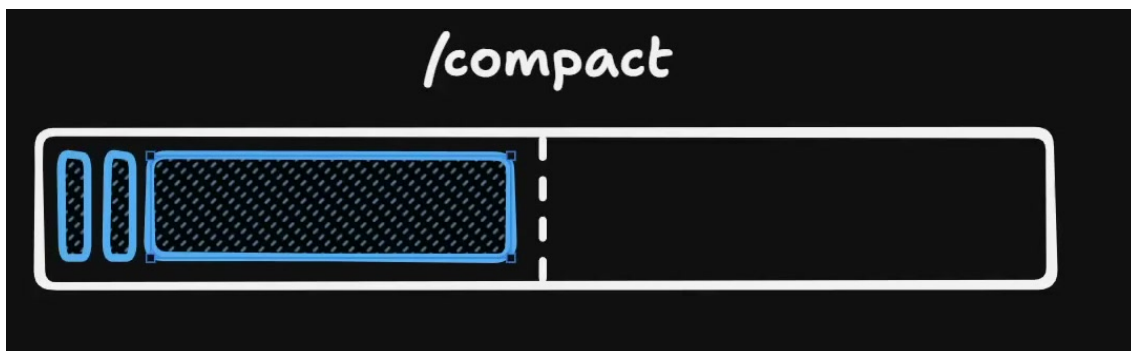


图 3: 画面显示 `/compact` 后的上下文条: 左侧有两个窄小蓝色摘要块, 旁边是一段新的蓝色压缩块, 整体仍位于虚线分界左侧。³

可以把 `compact` 想成“压缩续跑”。一个调试会话已经尝试了十几种方案, 原始日志和失败路径堆得很长; `compact` 把“试过什么、哪些文件相关、当前怀疑点是什么”压成摘要, 然后继续同一个问题。它减少了上下文体积, 但保留了继续工作所需的状态。

compact 的边界

`compact` 适合延长同一条主线: 同一个 bug、同一个 feature、同一个调试方向。它把长会话压成摘要后继续跑, 让代理从接近 `dumb zone` 的位置回到更靠近 `smart zone` 的位置。

Matt 还提到, 有些 harness 会有接近上下文上限时触发的自动 `compact` 缓冲机制: 当接近上下文窗口末端时, 系统可能自动触发总结, 把当前会话压进新会话 (03:34–03:49)。自动触发很方便, 但它也要求用户理解自己正在经历什么: 原始细节会被摘要取代, 摘要写得好不好会影响后续判断。

2.4 compact 的沉积层效应

`compact` 的强项也是它的代价。Matt 说, 当你继续 `compact`、再 `compact`, 会得到一种 `sediment`, 不同先前对话的层会堆在一起 (04:12–04:20)。这个词很形象: 河流把泥沙一层层压到河床, 长会话也会把一次次摘要压到后续会话的起点。

沉积层并非天然有害。调试时, 它能保存“已经试过的选项”, 避免代理每次从零开始 (04:30–04:43)。例如, 一个 bug 已经排除了配置问题、依赖版本问题、缓存问题; `compact` 后, 这些排除路径可以留在摘要里, 下一轮继续尝试新的方向。对于必须在同一个问题上反复推进的工作, `compact` 很有用。

但沉积层也会带来摩擦。摘要会丢失细节, 也会固化某些早期假设; 如果连续几次 `compact` 都把同一个误判写进开头, 它就像地图上的错误路标, 后续代理每次出发都会先看到它。更糟糕的是, 摘要层多了以后, 代理可能反复引用“旧会话说过什么”, 却不再容易区分哪些内容仍然成立。

compact 并非无损压缩

`compact` 更像工程摘要, 不能视作 `zip` 文件。它会保留重点, 也会丢掉上下文中的边角、犹豫、失败细节和隐含理由。连续 `compact` 后, 使用者要警惕摘要沉积带来的旧假设残留。

因此, `compact` 的正确位置很清楚: 它是长线续航工具。你还在做同一个问题, 只是原始会话太

³ 视频画面时间区间: 00:03:10–00:04:20。

长了，于是把状态压缩一下继续前进。下一章会进一步解释：当你想把旁支任务交给另一个会话，同时保持当前会话干净时，`handoff` 解决的是另一类边界问题。

2.5 本章小结

第二章定义了三件事。上下文窗口 C 是模型当前可见的活动 token 空间，它需要被预算。`smart zone` / `dumb zone` 是 Matt 对长会话质量变化的直观命名：负载低时更专注，负载高时更容易变散。`compact` 是把长会话总结成较小状态、继续同一条工作线的压缩续跑机制；它有用，尤其适合调试，但连续压缩会形成摘要沉积层。

3 `handoff` 与 `compact` 的真正分界

3.1 问题：旁支任务会污染主会话

Matt 在 04:49–06:14 讲的分界点很具体：他已经认可 `/compact` 对长单线会话有用，尤其适合调试这类需要把已经尝试过的路径保存下来的工作；可是他开始想把 `compact` “送到另一个会话”。这个愿望一出现，`compact` 的边界就暴露了。

视频里的例子是一次正在进行的编码会话。到 05:03–05:14，他说自己在当前会话中发现了一个小的 `refactoring opportunity`：这件事完全越界、超出当前目标范围，但迟早需要处理。这里的危险不在于重构本身有多难，危险在于它会把一个本来清晰的会话目标拆成两条并行思路。

如果继续留在原会话里做这个旁支，05:18–05:30 的后果是 `diluted context`：上下文被稀释，一半内容服务原目标，一半内容服务旁支修复。对代理来说，这就像在同一张工作台上同时摊开两个项目的图纸、日志、错误栈和计划。每一轮生成都要在更大的材料堆里辨认当前意图，前一章说的 `dumb zone` 会更早到来，原来的初始目标也更容易收不了尾。

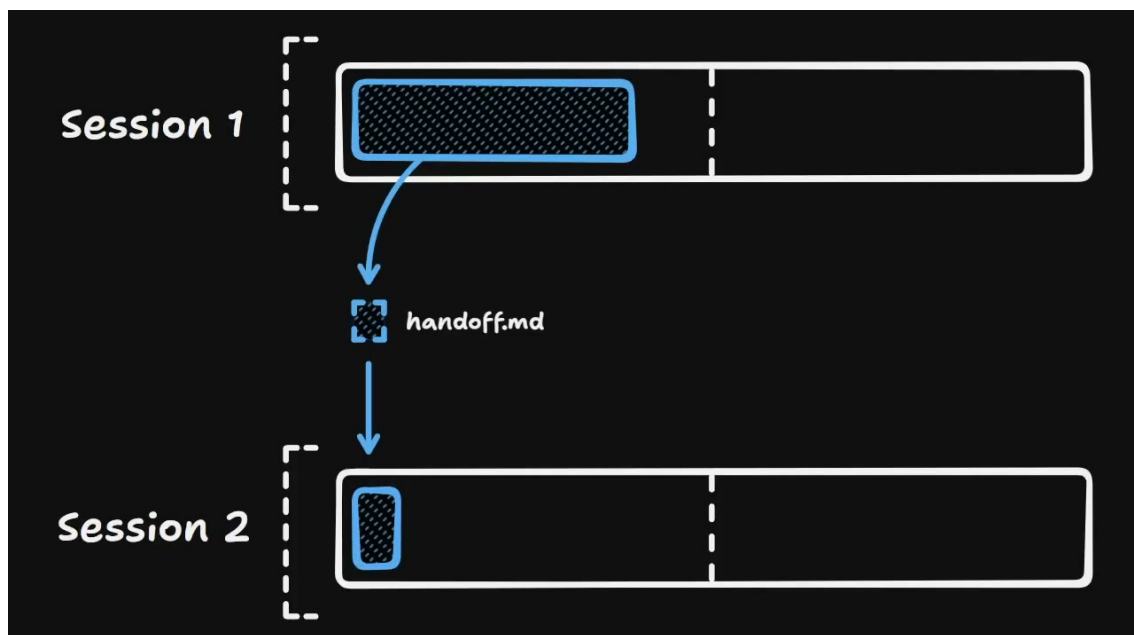


图 4: 画面显示两条标为 Session 1 和 Session 2 的会话上下文条：Session 1 左侧的蓝色任务块通过向下箭头写成 `handoff.md`，再由第二个向下箭头送入 Session 2 左侧的蓝色起始块。⁴

旁支任务会吞掉主线预算

旁支任务最容易被低估。它看起来只是一个小重构、一个顺手修复、一个以后会用到的 bug fix；进入当前会话后，它会占用同一个上下文窗口、同一个注意力预算、同一个任务轨道。真正的成本会落在主线目标上：判定条件开始变模糊。

3.2 compact 的局限：它会接管当前进度

Matt 给出的第二个选择是 compact。05:30–05:37 的说法很尖锐：如果选择 compact，他会 “clobber” 当前会话已经取得的进展。这里的 clobber 可以理解为覆盖或冲掉当前活动轨迹：系统把已有对话压成摘要，再让后续工作从这个摘要继续。这个动作适合继续同一个问题，却不适合把旁支任务拆到另一个执行轨道。

compact 的核心收益是续命：长会话快碰到上下文边界时，摘要把状态压回可工作的区域。它的隐含前提是“接下来仍然沿着同一条线走”。当用户真正想要的是“主线继续保持干净，旁支单独开工”，compact 就会把两种需求拧在一起：它让一个会话延续，却没有天然表达任务分叉。

这个边界可以用会话轨道来理解。compact 像把一列车的货物重新打包后继续装在同一列车上；handoff 像把其中一批货拆出来，贴上目的地和处理说明，交给另一列车。前者保留连续性，后者制造可控分支。

3.3 handoff 的机制：只切出相关切片

第 3 章需要把 handoff 形式化。先用普通话说清楚：当前会话不应该把全部上下文都捐给新会话。它应该只切出与旁支任务有关材料，再根据下一会话的意图写成一个可交接的 Markdown 文档。这个被切出的材料叫任务切片，后续会话拿到的是交接文档，而非整段历史。

$$H = \text{handoff}(C_{\text{task}}, I), \quad S' = \text{start}(H)$$

- C ：当前会话的上下文窗口，包含对话、工具结果、文件改动、错误信息、计划和已有摘要。
- C_{task} ：任务切片，也就是 C 中对旁支任务必要的那一部分。05:50–05:56 的 “the slice that pertains to this extra bug fix” 正对应这个概念。
- I ：下一会话的意图或焦点，例如 “修复这个特定 bug”、“为拆分 API 创建 GitHub issue”。
- H ：handoff 文档，通常是 `handoff.md` 这类 Markdown 文件，承载焦点、当前状态、相关指针和下一步动作。
- S' ：由 H 初始化的新会话。它从交接文档启动，不继承完整原始上下文。

这个公式的重点是参数。若输入是整个 C ，handoff 就会退化成一次笨重复制；若没有 I ，文档会像泛泛会议纪要，下一位代理不知道该优先做什么。真正有效的 handoff 同时具备切片和意图：切片控制信息量，意图控制解释方向。

⁴视频画面时间区间：00:05:45–00:06:10。

handoff 的正式定义

第 3 章的正式定义：**handoff** 是把当前上下文 C 中与某个意图 I 相关的任务切片 C_{task} 压缩为交接文档 H ，再用 H 启动新会话 S' 的工作方式。它管理的对象是任务分叉时的信息边界，单个长会话的续航则由 compact 这类压缩续跑机制承担。

3.4 两个会话并行：保持主线纯净

05:39–05:48 里，Matt 把自己的目标说得很直白：他想在单独会话里完成“另一件事”，同时 keep my current session pure。这里的 pure 不需要神秘化，意思是主会话继续服务原始目标，旁支会话只处理额外 bug fix 或重构任务。

到 05:58–06:10，他描述了手工做法：让当前会话收集相关材料，写一个 `handoff.md`，然后把它传给另一个 agent。这个动作频繁到他决定做成 skill。换句话说，handoff skill 的价值来自重复出现的边界管理需求：每次用户想把一块问题分离出去时，都需要一个稳定的交接格式。

新会话边界

“新会话边界”是这里的隐藏关键。它远不止换一个聊天窗口这么表面；它让下一轮推理拥有独立的上下文预算、独立的任务完成条件和独立的失败记录。原会话保留主线判断，新会话承担旁支执行。

这个模式也解释了为什么 handoff 文档应当短命、聚焦、可丢弃。它的身份更像快递面单加装箱清单：把该交给下一位处理者的信息交清楚，任务完成后不必成为项目永久文档。若把它写成百科式总结，反而会把 compact 的上下文膨胀问题搬进新会话。

3.5 本章小结

compact 和 handoff 都在处理上下文预算，但边界不同。compact 适合把一个长会话压缩后继续同一条工作线；handoff 适合把一个旁支任务切出来，让新会话独立完成。Matt 的 refactoring opportunity 例子说明了真正的成本：旁支任务留在主会话会制造 diluted context，选择 compact 又可能 clobber 当前进度。好的 handoff 用 C_{task} 、 I 、 H 、 S' 四个对象把这件事拆清楚：取相关切片，声明下一会话意图，写成交接文档，再启动新的会话边界。

4 在 grilling 中把旁支问题移出去

4.1 grilling 是什么

第 4 章的核心场景从 06:14 开始。Matt 说他最常在 grilling 时使用 handoff。这里的 grilling 指一种由代理主导的追问式规划：代理连续提出问题，把需求、约束、未决选择和范围边界逼到台面上。它区别于闲聊式 brainstorming，更接近代码评审前的需求盘问，目标是让含糊的计划经得住执行。

在这种模式里，用户通常还没有完整 PRD，代理也不能直接开工。代理通过问题把“已知未决项”一个个拉出来：目标是什么，不能做什么，验收标准是什么，是否有依赖，哪些地方需要另开任务。grilling 的价值就在于它把隐含约束显性化。

grilling 的工作定义

本章定义：**grilling** 是一种追问式规划会话，代理通过结构化问题迫使需求、约束、范围和未决决策显性化。它的产物不一定是代码，常见产物是更清楚的范围、PRD、issue、设计取舍或后续任务清单。

这个定义也说明了为什么 grilling 容易遇到 handoff 场景。追问越有效，越容易发现旁支问题；旁支问题若立刻展开，会把规划会话拖成执行会话。handoff 的作用是给旁支一个出口，让当前问题继续收束。

4.2 一次 Q2 里的 handoff

视频里的例子来自 Sand Castle, Matt 称它是自己的 software factory。06:17–06:30，他正在为未来功能做规划，而且只进行到 grilling session 的 Q2，说明会话还处在很早的澄清阶段。

06:30–06:41 的关键内容是：他认为未来可能要把 iterations 和 completion signal 移到一个 separate API 上，于是直接说把这个任务 hand off 给 separate agent。这里没有等到整个 grilling 做完，也没有把 API 拆分问题塞进 Q2 继续讨论。旁支一出现，Matt 就把它包装成单独任务。

```
● Write(/tmp/handoff-J1D7KN.md)
  └ Wrote 57 lines to ../../../../tmp/handoff-J1D7KN.md
    2 # Handoff: split iterations + completion signal off `run()`
    3 ## Focus for the next session
    4
    5 File a GitHub issue (and eventually design) for splitting iterations and
completion signal into a separate API from the core `run()`. The constraint
    comes from incoming work on structured output (issue #517), which we
    to `maxIterations === 1` so that an iteration/loop API can e
    volve later without breaking structured-output callers.
    6
    7 This handoff is not about implementing structured output. It is about
    at protects it.
    8
    9 ## Why this is now on the table
    ... +56 lines (ctrl+o to expand)

● Handoff written to /tmp/handoff-J1D7KN.md.

Returning to the #517 grill – still waiting on your confirmation for:
```

图 5：终端画面显示 Write(/tmp/handoff-J1D7KN.md) 已写入 57 行；文件标题为 Handoff: split iterations + completion signal off `run()`，焦点段落要求提交 GitHub issue，并把 iterations 与 completion signal 从核心 run() 中拆到单独 API，画面中 separate API from the core `run()` 被高亮。⁵

这个动作有三个组成部分。第一，指出旁支对象：iterations 与 completion signal。第二，指出目标形态：separate API。第三，指出执行载体：separate agent。三者合起来，下一会话拿到的是一个能开工的任务，而非一句模糊提醒。

⁵ 视频画面时间区间：00:06:30–00:07:20。

iterations 与 completion signal

iterations 可以理解为系统执行一轮又一轮工作的循环单元；completion signal 是判断“这一轮或这一项已经完成”的信号。把二者拆到 separate API，意味着把流程控制和完成判定从原系统里抽出成更清楚的接口边界。这个决定会影响架构，不能顺手塞进一个 Q2 答案里草草处理。

4.3 为什么说明“交出去的原因”很关键

06:41–06:51 里，Matt 强调了一个细节：他在定义 handoff 时，会说明为什么要交出去，以及文档里应该包含什么。这是好 handoff 和普通摘要的分水岭。普通摘要回答“刚才发生了什么”；handoff 还要回答“下一位代理为什么接手、接手后该完成什么”。

在 Sand Castle 例子里，交出去的原因是范围控制：当前 grilling 正在回答 Q2，API 拆分是一个独立设计任务。文档需要包含下一会话的焦点，即为拆分 iterations 与 completion signal 到 separate API 先 file a GitHub issue，再进一步设计。07:05–07:16 的字幕说明，生成的 Markdown 文件写明了这个 focus for the next session。

交出去的原因要写清楚

handoff 文档缺少“为什么交出去”时，下一会话很容易误判优先级。它可能把旁支当成背景资料，也可能把背景资料当成主任务。原因说明相当于任务的路标：告诉新代理这份文档为何存在，以及完成后应当回收什么结果。

这种原因说明还保护当前会话。用户一旦明确“这件事交出去”，当前 grilling 就不必继续为 separate API 展开细节。范围收缩并不等于少想问题；更准确地说，它是在把问题放到合适的执行容器里。

4.4 范围收缩带来的反作用

06:51–07:03 的一句话很有教学价值：Matt 说这个动作会 sharpen 当前 grilling session，并出现“given that constraint, Q2 collapses”。Q2 collapses 的意思是，一旦把 API 拆分移出当前范围，Q2 本身变简单了。它不再需要同时承担原规划问题和架构拆分问题。

这就是 handoff 在 grilling 中的反作用：它帮助下一会话，也反过来清理当前会话。旁支被移走后，当前代理可以继续沿着原问题追问，不会因为一个大设计岔路而偏航。换成规划语言说，handoff 是一种范围收缩工具；换成上下文语言说，handoff 是一种注意力隔离工具。

这里还有一个容易忽略的收益：handoff 让“以后再说”变得可执行。普通的以后再说只是一句搁置；带有 Markdown 文档、明确焦点和后续动作的 handoff 会变成一个可传递任务。Matt 后面说，他会把这个文档传给另一个 agent 去创建 issue。于是旁支没有消失，它只是离开了当前 grilling 的上下文窗口。

handoff 对当前会话的反作用

grilling 中的 handoff 同时服务两个方向：对下一会话，它提供任务焦点和必要上下文；对当前会话，它把旁支标记为 out of scope，让原问题继续收束。一个好的 handoff 会让接收方更清楚，也让发送方更轻。

4.5 本章小结

grilling 是代理通过结构化追问逼出需求和约束的规划方式。Matt 在 Sand Castle 的 Q2 中发现 iterations 与 completion signal 可能需要拆到 separate API，于是把该任务交给 separate agent，并要求 handoff 文档写明原因、焦点和下一步。这个例子说明，handoff 在 grilling 中的主要价值不只是让别人接着做事，还能让当前问题收缩：旁支离开上下文窗口，Q2 变得更简单，规划会话重新聚焦。

5 prototype 往返：手搓一个轻量 subagent

到 07:20 以后，Matt 把 handoff 从“把旁支任务交给另一个会话”推进到一个更强的模式：规划会话先把不确定的实现问题交给原型会话，原型会话用自己的上下文预算探索代码、UI 或逻辑，再把非显而易见的学习结果压缩回规划会话。这个往返使 handoff 不只是向外分流任务，也能把外部探索变成可回收的知识。

5.1 known unknowns 与必须看见的东西

Matt 在 07:25–07:45 给出第一个分界：在 grill-me 或 grill-with-docs 这类追问式规划中，问题大致分成两类。第一类是 *known unknowns*，也就是“已经知道自己不知道”的问题。它们像需求表里的空格：代理可以直接问用户，例如“这个 API 的完成信号由谁发出？”、“错误状态要不要显示重试按钮？”。

第二类更麻烦。它们表面上也像问题，实际需要通过代码、界面或复杂逻辑跑一遍才会显形。07:42–07:53 里 Matt 说，有些东西需要“在代码里看到”或“被 prototype 出来”，尤其是 UI 原型和还不知道如何处理的复杂逻辑。这里的关键差异在于信息来源：*known unknowns* 的答案通常来自用户或已有文档；原型类问题的答案来自一次受控实验。

known unknowns 的边界

known unknowns 可以通过提问收敛；原型类不确定性需要通过运行、集成、渲染或交互收敛。前者像填表，后者像做实验。把实验塞进同一个规划会话，会让规划上下文被大量实现细节占满。

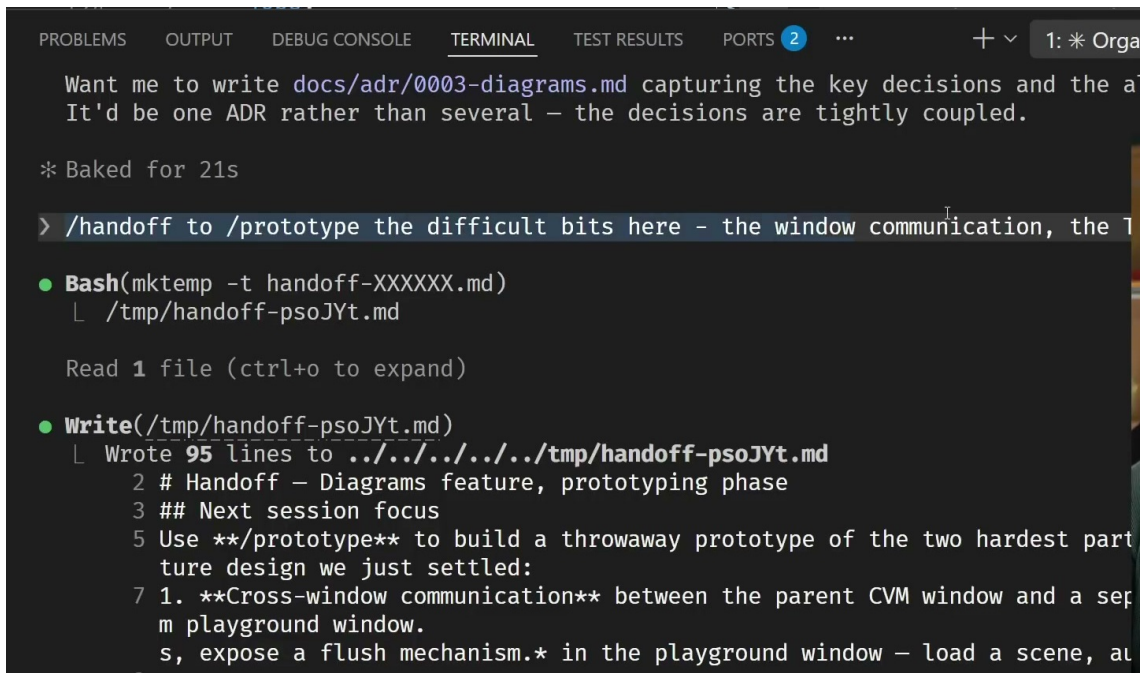
这也是为什么 handoff 在规划中有特殊价值。规划会话的任务是维持问题结构，原型会话的任务是暴露真实约束。二者共用一个上下文窗口时，规划信息和实验日志会互相挤压；分成两个会话后，各自都能保持锋利。

5.2 把困难部分交给 prototype

本章中的 *prototype* 指临时实现：它可以是 UI 草图、交互 spike、SDK 集成试验，也可以是一段复杂逻辑的最小可运行版本。它的价值不在于直接交付生产代码，而在于把“脑内推测”变成“可观察现象”：什么 API 真能用、哪些状态会冲突、哪个交互路径在屏幕上看起来别扭、哪段逻辑会制造边界条件。

Matt 的例子发生在 08:03–08:15。他在第 13 个 grilling 问题之后说，把困难部分 handoff to prototype：具体困难包括窗口通信，以及当时正在做的 tldraw SDK（按画面和上下文规范化的产品名）集成。随后 handoff 生成文档，他到那个分支上实现原型。

08:20–08:29 的补充让 prototype 的定位更清楚：Matt 是在对应分支上观察 UI 与 interaction 是否符合预期。也就是说，prototype 的输出不只是一段说明，它要把界面行为、窗口通信和 SDK 集成变成可以运行、可以看见、可以回传给 planner 的证据。



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TEST RESULTS  PORTS 2  ...  + v  1: * Orga

Want me to write docs/adr/0003-diagrams.md capturing the key decisions and the a
It'd be one ADR rather than several – the decisions are tightly coupled.

* Baked for 21s

> /handoff to /prototype the difficult bits here - the window communication, the 1

● Bash(mktemp -t handoff-XXXXXX.md)
  └─ /tmp/handoff-psoJYt.md

Read 1 file (ctrl+o to expand)

● Write(/tmp/handoff-psoJYt.md)
  └─ Wrote 95 lines to ../../../../tmp/handoff-psoJYt.md
      2 # Handoff – Diagrams feature, prototyping phase
      3 ## Next session focus
      5 Use **/prototype** to build a throwaway prototype of the two hardest part
      6 ture design we just settled:
      7 1. **Cross-window communication** between the parent CVM window and a sep
      8 m playground window.
      9 s, expose a flush mechanism.* in the playground window – load a scene, at
```

图 6: 在 prototype 示例中，终端里出现了 `/handoff to /prototype` 命令，目标被写成“the difficult bits here”，并明确点出 window communication 等需要通过原型暴露的难点；随后系统生成了位于 `/tmp/` 下的 handoff 文档。⁶

prototype 的工作定义

prototype 是一段短命的、为学习服务的实现。它的判断标准应放在学习收益上：能不能暴露原先看不见的行为和约束。能否直接上线只是次要问题。在 Matt 的例子中，窗口通信和 `tldraw` SDK（按画面和上下文规范化的产品名）集成本都属于必须通过实现才能看清的部分。

这个选择有一个很硬的上下文预算理由。08:14–08:22 里 Matt 说，原型会话最后变成了一个 169K tokens 的大 session，远大于能舒适塞回 grilling 的体量。这个数字说明原型需要大量具体操作：它会产生文件编辑、工具调用、错误信息、界面观察和修正路径。把这些全部放进规划会话，会提前把规划推向 Section 2 里说过的重上下文区域。

5.3 从 prototype 再 handoff 回 planner

原型完成后，Matt 没有让规划会话直接阅读完整原型会话。他在 08:29–08:45 描述了回程：把这个原型 handoff 回生成它的 grilling session，取出所有 prototype learnings，尤其是没有直接被原型代码捕获、或者即使在代码里也不明显的部分，写成一个可以传回 planner 的 handoff 文档。

这里出现了一个常见误区：很多人会以为“代码已经在分支上，所以规划会话只要看代码就够了”。实际工程中，原型的高价值信息经常藏在代码之外。例如：

⁶视频画面时间区间：00:08:00–00:08:10。

- 某个 SDK 的文档看起来支持一种调用方式，但实际运行需要额外初始化顺序；
- UI 交互在静态组件里可行，放进窗口通信后出现状态同步延迟；
- 一个方案能跑通，但实现者在调试中发现它会让后续 PRD 的范围扩大；
- 原型代码里有临时 hack，planner 需要知道哪些结论可靠、哪些只是为了快速验证。

回程 handoff 不能只说“原型完成了”

回传给 planner 的文档应当写出非显而易见的学习结果：踩过的坑、被证伪的路径、仍需产品决策的点、以及原型代码本身无法表达的实现约束。否则父会话会得到一个空洞状态更新，仍然无法完成 PRD 或 issue。

可以把这个往返写成一个很小的公式。直觉上，当前规划会话先抽出原型任务切片，形成 handoff 文档；新会话由这个文档启动；新会话完成实验后，再产生学习摘要回到父会话。

$$H_p = \text{handoff}(C_{\text{proto}}, I_p), \quad S' = \text{start}(H_p), \quad H_r = \text{return}(S', L_{\text{proto}})$$

- C_{proto} : 当前规划会话中与原型任务相关的上下文切片，例如窗口通信、tldraw SDK（按画面和上下文规范化的产品名）集成、当前问题编号和目标分支。
- I_p : 原型会话的意图，例如“请验证这些困难部分，规划主线先留在父会话”。
- H_p : 发给原型会话的 handoff 文档。
- S' : 由 H_p 启动的新会话，也就是本章定义的轻量子会话。
- L_{proto} : 原型过程中得到的学习结果，包括代码能表达的结果和代码外的观察。
- H_r : 回传给 planner 的 handoff 文档，它把 S' 中的关键学习压缩回父会话。

5.4 DIY subagent 的结构

Matt 在 08:47–09:08 把这个模式称为“almost like you’ve done a kind of DIY sub agent”。这里的 *subagent* 指由 Markdown 文件和新会话手工拼出的轻量结构，区别于平台原生的子代理功能。

这个结构有四个角色：

- **父会话**: 最初的 planning 或 grilling 会话，掌握产品目标、问题序列和整体方向。
- **出站 handoff**: 父会话写给原型会话的短文档，规定实验焦点和必要上下文。
- **子会话 S'** : 从 handoff 文档启动的新 session，使用自己的上下文窗口专门处理一个任务。
- **回程 handoff**: 子会话把学习结果压缩给父会话，帮助父会话继续生成 PRD、issue 或计划。

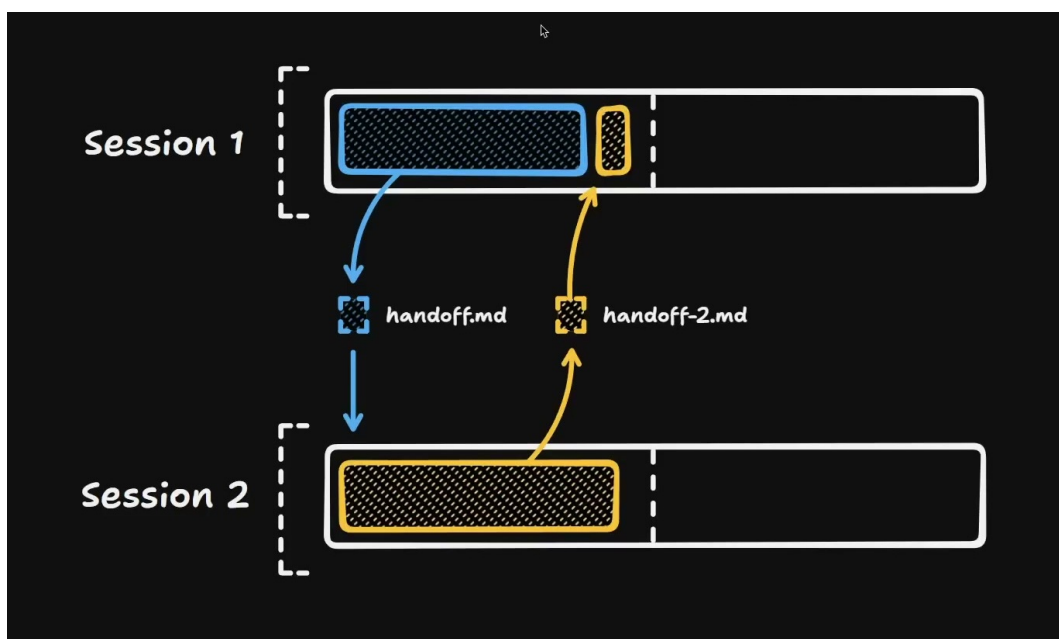


图 7: 画面中 Session 1 通过蓝色的 `handoff.md` 把任务交给 Session 2, Session 2 完成原型探索后再通过黄色的 `handoff-2.md` 把学习结果压回给 Session 1。⁷

本章对 S' 的定义

S' 是由 handoff 文档 H 初始化的新代理会话。它继承父会话为一个明确任务挑出的上下文切片和执行意图；完整父会话上下文不进入这个新会话。把 S' 看成“短命任务工作台”更准确：它可以很大、很忙、很脏，但回到父会话的内容必须压缩、可读、可执行。

09:08–09:18 里，Matt 说自己随后能够完成 grilling session，并生成 proper PRDs and issues，而且其中包含 prototype 的结果。这一句很重要：原型会话没有替代规划会话，它给规划会话补上了实践证据。最终产物仍然是规划产物，只是它们不再停留在猜测层。

5.5 跨代理可移植性

09:23–09:50 的补充把这个模式从“同一种工具内的子会话”扩展成“跨代理 workflow”。Matt 强调，使用 Markdown 文档的好处在于不依赖原生 agent 功能：第一个会话可以在 Claude Code 中，后续可以交给 Codex、Copilot CLI 或其他编码代理。自动字幕把这些名字识别得不稳定，但语义很清楚：Markdown 是最低公共接口。

这种可移植性给了两个实用能力。第一，用户可以让不同代理在同一任务链里接力，例如一个代理做原型，另一个代理做 review。第二，用户可以做 adversarial review，也就是让不同工具用不同偏好和失败模式检查同一个结果。由于交接物是普通文本，workflow 不被某个产品的内置子代理模型锁住。

Markdown 作为最低公共接口

原生 subagent 通常更自动，代价是依赖某个 harness 的运行。Markdown handoff 更笨，也更透明：文件里写了焦点、上下文、指针和下一步，任何能读文本的代理都能接上。这种透明

⁷ 视频画面时间区间：00:09:05–00:09:15。

性对跨工具协作尤其有用。

5.6 本章小结

本章的核心机制是一个受控反馈环：父会话抽出原型任务，生成 handoff；新会话 *S'* 用独立上下文做实验；原型会话再把非显而易见的学习结果 handoff 回 planner。Matt 的 169K-token 原型例子说明，原型工作会迅速膨胀，强行塞进 planning 会话会破坏上下文预算。把它作为一个临时子会话处理，既保留实验空间，也让父会话拿到足够压缩的证据来完成 PRD、issue 和后续规划。这也解释了它为什么适合 AFK agents：子会话可以独立跑很久，父会话只接收压缩后的学习结果。

6 技能设计：让 handoff 文档短命、聚焦、可执行

09:50 以后，Matt 开始逐条读 handoff 技能末尾的设计规则。这里的主题从“怎么用 handoff”转到“为什么这个 skill 要这样写”。本章中的 *skill design* 指把可重复的代理行为封装成明确指令：它要告诉代理产出什么、存到哪里、避免什么、如何根据用户参数收缩焦点。好的 skill 像一份很小的操作规程，短到能被执行，硬到能阻止常见坏习惯。

6.1 suggested skills: 给下一位代理预装工作方式

09:58–10:25 里，Matt 解释第一个规则：handoff 文档应包含 **suggested skills** section，建议下一位代理应该调用哪些 skills。他给出的例子包括 grill-with-docs、diagnose 和 prototype。这类元数据的实际作用，是给下一段会话预装工作方式。

原因很直接：同一个 handoff 文档可能被粘贴到一个全新的 session。新代理拿到上下文切片后，还需要知道自己应该采用什么工作模式。若目标是继续追问需求，它应该进入 grilling；若目标是定位失败，它应该进入 diagnose；若目标是做临时实现，它应该进入 prototype。没有 suggested skills，新会话会花额外轮次猜测“接下来该怎么干”。

```

skills > productivity > handoff > SKILL.md
1 ---
2 name: handoff
3 description: Compact the current conversation into a handoff
4 argument-hint: "What will the next session be used for?"
5 ---
6
7 Write a handoff document summarising the current conversation so
8 a fresh agent can continue the work. Save to the temporary
9 directory of the user's OS - not the current workspace.
10
11 Include a "suggested skills" section in the document, which
12 suggests skills that the agent should invoke.
13
14 Do not duplicate content already captured in other artifacts
15 (PRDs, plans, ADRs, issues, commits, diffs). Reference them by
16 path or URL instead.
17
18 Redact any sensitive information, such as API keys, passwords,
19 or personally identifiable information.
20
21 If the user passed arguments, treat them as a description of
22 what the next session will focus on and tailor the doc
23 accordingly.
24

```

图 8: SKILL.md 中的 handoff 规则同时约束了文档位置、建议技能、重复内容、敏感信息脱敏和参数用途：这些规则让 handoff 文档保持短命、聚焦、可执行。⁸

suggested skills 的作用

suggested skills 把“任务内容”和“执行风格”绑定在一起。handoff 只告诉新会话要处理什么还不够；它还要提示新会话用哪一种技能范式处理，否则同一份上下文可能被用成规划、调试、原型或审查四种完全不同的任务。

这里也能看出 handoff 文档与普通摘要的差别。普通摘要偏向“发生过什么”；handoff 文档必须面向“下一位代理要怎么行动”。它不追求记述完整历史，而追求让新会话少走弯路。

6.2 用指针替代重复内容

10:28–10:48 的第二条规则是不要复制已经存在于其他 artifact 里的内容。Matt 说，他经常发现 handoff 文档变得很大，因为它们重复了其他 Markdown 文件、GitHub issues 或资源里的内容。所以技能要求使用 pointers，也就是指针或引用线索。

指针纪律可以理解为“保留 source of truth”。如果 PRD 已经在 docs/prd.md，handoff 里应写“参考 docs/prd.md 的某节”，避免把整段 PRD 再复制一遍。如果 bug 细节已在 GitHub issue，handoff 应给 issue 编号、链接、相关评论或 commit 范围。这样做有三点收益：

- handoff 文档保持短小，新会话能快速进入任务；
- 原始 artifact 仍是唯一事实来源，后续修改不会造成两份文本互相矛盾；
- 敏感信息和噪声更少被复制到一次性 Markdown 中。

⁸视频画面时间区间：00:10:45–00:10:55。

复制内容会制造文档腐烂

一份 handoff 如果把 PRD、issue、diff 和日志都复制进去，很快会变成第二套项目文档。第二套文档通常没人维护，过几天就开始腐烂。指针纪律的底线是：能引用已有 artifact 的地方，优先引用；只有下一会话必须立即使用的最小摘录才进入 handoff。

用信息论的直觉说，handoff 文档要降低新会话的搜索成本，文本体积本身没有价值。它应该保存“入口”和“决策相关状态”，避免把整个项目搬进临时文件。

6.3 临时目录与信息安全

10:50–11:04 的第三条规则是存储位置：Matt 强烈主张把 handoff 文件保存到用户操作系统的临时目录。理由也在字幕里明说：这些文件是 disposable，不应长期留在代码库里，慢慢变成腐烂文档。

这条规则的反面概念是 workspace pollution，即工作区污染。代理很容易把中间计划、临时总结、调试手稿写进 repo；短期看方便，长期看会让仓库里出现大量无人负责的半成品文档。临时目录给了 handoff 正确的生命周期：它只为下一段会话服务，完成交接后就失去存在价值。

紧接着 11:01–11:12, Matt 提到 redaction：要脱敏 sensitive information，包括 API keys、passwords 和 PII。这里的风险比“文档变乱”严重得多。handoff 文档会被复制、粘贴、上传、传给另一个代理或另一个工具；如果里面含有密钥或个人信息，它就从一次性工作文件变成泄露载体。

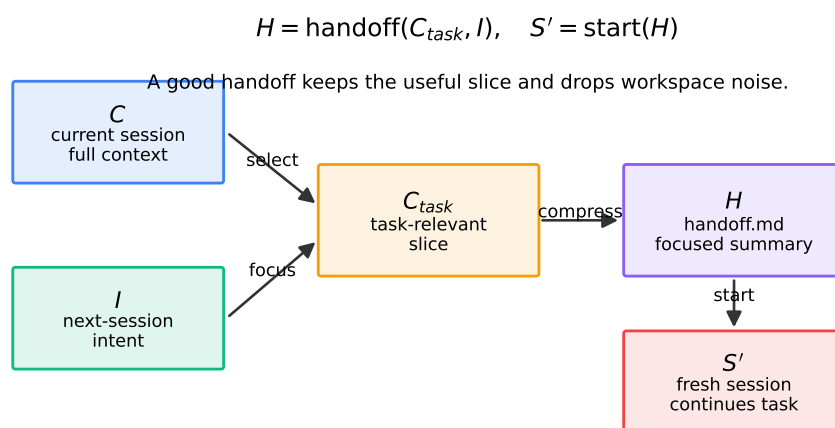


图 9: 重绘的 handoff 机制图：当前会话上下文 C 先被切成任务相关的 C_{task} ，再结合下一会话意图 I 压成交接文档 H ，最后启动新会话 S' 。

handoff 是传播性文件

handoff 的用途决定了它会移动：从父会话到子会话，从一个代理到另一个代理，从终端到编辑器。所有会移动的文件都要默认做脱敏。API key、password、token、cookie、个人邮箱、手机号、客户数据和内部 URL 都需要在写入前被替换、概括或指向受控来源。

临时目录和脱敏是一组互补规则。临时目录解决生命周期问题：文件不应长期留在 repo。脱敏解决传播风险：文件即使短命，也不能携带不该扩散的信息。

6.4 参数即下一会话的焦点

11:14–11:40 的最后一条规则处理用户传入的 arguments。Matt 解释，如果用户传了参数，也就是说明下一段会话将用于什么，就要把这些参数当作下一会话的 focus，并据此裁剪文档。他认为这是 handoff 的核心，因为代理必须知道下一段 agent session 要关注什么，才可能写出 decent document。

这里的参数 I 可以来自一句很自然的话，例如：

- “把窗口通信和 tldraw SDK（按画面和上下文规范化的产品名）集成交给 prototype”；
- “为这个 separate API 任务生成 GitHub issue”；
- “把原型里的非显而易见学习结果 handoff 回 planner”；
- “让另一个代理做 adversarial review”。

参数是裁剪函数的目标，地位高于普通附加说明。没有 I ，代理只能写宽泛总结；有了 I ，代理才能决定哪些上下文进入 H ，哪些只需要指针，哪些必须省略。

先用普通话说：好的 handoff 会按下一会话的焦点压缩当前上下文，避免按时间顺序复述当前会话。可以把它写成：

$$H = \text{select_and_compress}(C, I)$$

- C ：当前会话可见的完整上下文，包括消息、工具结果、文件状态和已经形成的判断。
- I ：下一会话的焦点或用途，通常由用户在调用 handoff 时说明。
- H ：最终 handoff 文档，它只保留服务于 I 的必要信息、指针和下一步。
- select_and_compress：选择并压缩的过程，先按焦点筛选，再把筛出的内容写成可执行文档。

Matt 还提到 dictation 会让这件事更容易：他说自己可以直接把目的口述出来。这里的重点放在目的外显上，语音输入只是便利手段。用户把“为什么 handoff、交给谁、做什么”说清楚，文档才会聚焦。

参数是 handoff 的准星

如果用户只说“写个 handoff”，代理容易生成泛泛摘要；如果用户说“写给 prototype 会话，专门验证窗口通信和 tldraw SDK（按画面和上下文规范化的产品名）集成”，代理就能收缩上下文、建议合适技能、保留关键指针，并省掉无关历史。

6.5 总结与延伸

11:45–11:53 的实质性收束很短，但信息密度高：Matt 说，这就是 handoff，它已经成为自己工具箱里的 essential skill，而且和很多其他 skills 一样，几周前还不存在。后面关于 cohort course、观看感谢和书架的内容主要是频道收尾；对本课有价值的是这句“最近才出现，却很快变成必需品”。

这说明一个现实：代理 workflows 的关键能力不一定来自庞大框架。很多时候，真正改变日常效率的是一个小而硬的协议。这个协议规定交接物是什么、放在哪里、怎么脱敏、如何引用已有资料、下

一会话要用什么技能、目标参数如何收缩内容。它小到可以被粘贴进任何 session，大到足以改变上下文预算的使用方式。

从全片看，`handoff` 串起了三层机制：

- **上下文预算机制：**长会话会变重，`compact` 适合延续同一条主线，`handoff` 适合把旁支任务切出去。
- **任务分叉机制：**父会话保留规划目标，子会话 *S'* 专门处理 bug、prototype、issue 或 review，再把学习结果压缩回来。
- **文本协议机制：**Markdown `handoff` 让 Claude Code、Codex、Copilot CLI 等不同代理通过同一份显式文档协作。

最容易被低估的是第三层。人们常把 agent collaboration 想成复杂调度系统，但 Matt 的方案先用一个普通 Markdown 文件建立边界。边界越清楚，自动化越容易：父会话不需要知道子会话将怎样完成任务；子会话不需要继承父会话的所有历史；回程 `handoff` 只需要交回有用学习。

落到实践，读者可以把 `handoff` 当作四问模板：

- 下一会话的焦点是什么？
- 它需要哪些最小上下文和哪些外部指针？
- 它应调用哪些 suggested skills？
- 哪些内容必须留在临时文件之外，尤其是密钥、密码和 PII？

如果这四个问题答不清，`handoff` 文档就会退化成“长摘要”。如果四个问题答得清，它会成为一个短命、聚焦、可执行的工作包。Matt 这支视频的核心价值也在这里：它把 agent 协作从神秘化表达拉回可检查的交接协议。谁负责父会话，谁负责子会话，哪些信息通过 *H* 传递，哪些信息只用指针引用，哪些信息严禁进入文档，这些边界一旦明确，用户就能把原型、审查、调试和规划组织成多会话反馈环。